

# IBM Business Automation Manager Open Editions Demos and Labs 2026

## Model your first Workflow and DMN with BAMOE

V 2.0 (for IBM BAMOE 9.5.2)

Raul Mariano  
[raul.mariano@ibm.com](mailto:raul.mariano@ibm.com)

## NOTICES

This information was developed for products and services offered in the USA.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing  
IBM Corporation  
North Castle Drive, MD-NC119  
Armonk, NY 10504-1785  
United States of America

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

## TRADEMARKS

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at "Copyright and trademark information" at [www.ibm.com/legal/copytrade.shtml](http://www.ibm.com/legal/copytrade.shtml).

Adobe, the Adobe logo, PostScript, and the PostScript logo are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, and/or other countries.

Cell Broadband Engine is a trademark of Sony Computer Entertainment, Inc. in the United States, other countries, or both and is used under license therefrom.

Intel, Intel logo, Intel Inside, Intel Inside logo, Intel Centrino, Intel Centrino logo, Celeron, Intel Xeon, Intel SpeedStep, Itanium, and Pentium are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

IT Infrastructure Library is a Registered Trade Mark of AXELOS Limited.

ITIL is a Registered Trade Mark of AXELOS Limited.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

Linear Tape-Open, LTO, the LTO Logo, Ultrium, and the Ultrium logo are trademarks of HP, IBM Corp. and Quantum in the U.S. and other countries.

Linux is a registered trademark of Linus Torvalds in the United States, other countries, or both.

Microsoft, Windows, Windows NT, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

UNIX is a registered trademark of The Open Group in the United States and other countries.

© Copyright International Business Machines Corporation 2024.

This document may not be reproduced in whole or in part without the prior written permission of IBM.

US Government Users Restricted Rights - Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

# Table of Contents

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction.....</b>                                                      | <b>4</b>  |
| 1.1      | IBM Business Automation Manager Open Editions.....                            | 4         |
| <b>2</b> | <b>Lab Setup Instructions .....</b>                                           | <b>5</b>  |
| 2.1      | Access the environment.....                                                   | 5         |
| 2.1.1    | About the Environment: .....                                                  | 5         |
| <b>3</b> | <b>The use case Lab .....</b>                                                 | <b>7</b>  |
| <b>4</b> | <b>Exercise 1: Create a DMN using BAMOE Canvas .....</b>                      | <b>8</b>  |
| <b>5</b> | <b>Exercise 2: Create a workflow using BAMOE Canvas.....</b>                  | <b>15</b> |
| <b>6</b> | <b>Exercise 3: Run your project using VS Code .....</b>                       | <b>28</b> |
| <b>7</b> | <b>Exercise 4: Connect your project to the BAMOE Management Console .....</b> | <b>37</b> |
| <b>8</b> | <b>Consult Documentation and Communities .....</b>                            | <b>38</b> |

# 1 Introduction

These hands-on lab exercises are designed to guide you through the essential aspects of process automation using IBM BAMOE Development Tools for VS Code. Whether you're a developer or an architect, these labs will provide the skills needed to use BAMOE Canvas and VS Code to effectively create workflow automation and decision-making designs in modern, cloud-native business automation solutions.

## Notes:

- *Includes **four** exercises. We recommend completing them sequentially.*
- **Duration:** *Approximately 2 hours.*
- **Target audience:** *Anyone who wants to learn how to use IBM Business Automation Manager Open Editions.*

## 1.1 IBM Business Automation Manager Open Editions

IBM Business Automation Manager Open Editions (IBM BAMOE) is a powerful open-source solution that serves as a foundation platform for tailoring long-lasting business automation solutions for the hybrid cloud.

With a developer-centric approach, this comprehensive and flexible platform makes it easy for teams to collaborate through Open Standards and efficient development tools suited for different personas.

Each automation solution can be shaped to perfectly address each scenario: business applications are flexible and can effortlessly integrate with external systems of your existing architecture.

Designed for the hybrid cloud, IBM Business Automation Manager Open Editions, accelerates the application modernization and cloud adoption journeys, as the lightweight design tools, business applications and other product components can be containerized and deployed with popular technologies such as Kubernetes and OpenShift.

For more information, see IBM documentation and other useful links:

- [IBM Business Automation Manager Open Editions Documentation](#)
- [Open Editions Community](#)

## 2 Lab Setup Instructions

### 2.1 Access the environment

You received this email with instructions on how to access the environment using your IBMid. Otherwise, click [here](#) to reserve it.

**If necessary, this is the Windows credential:**

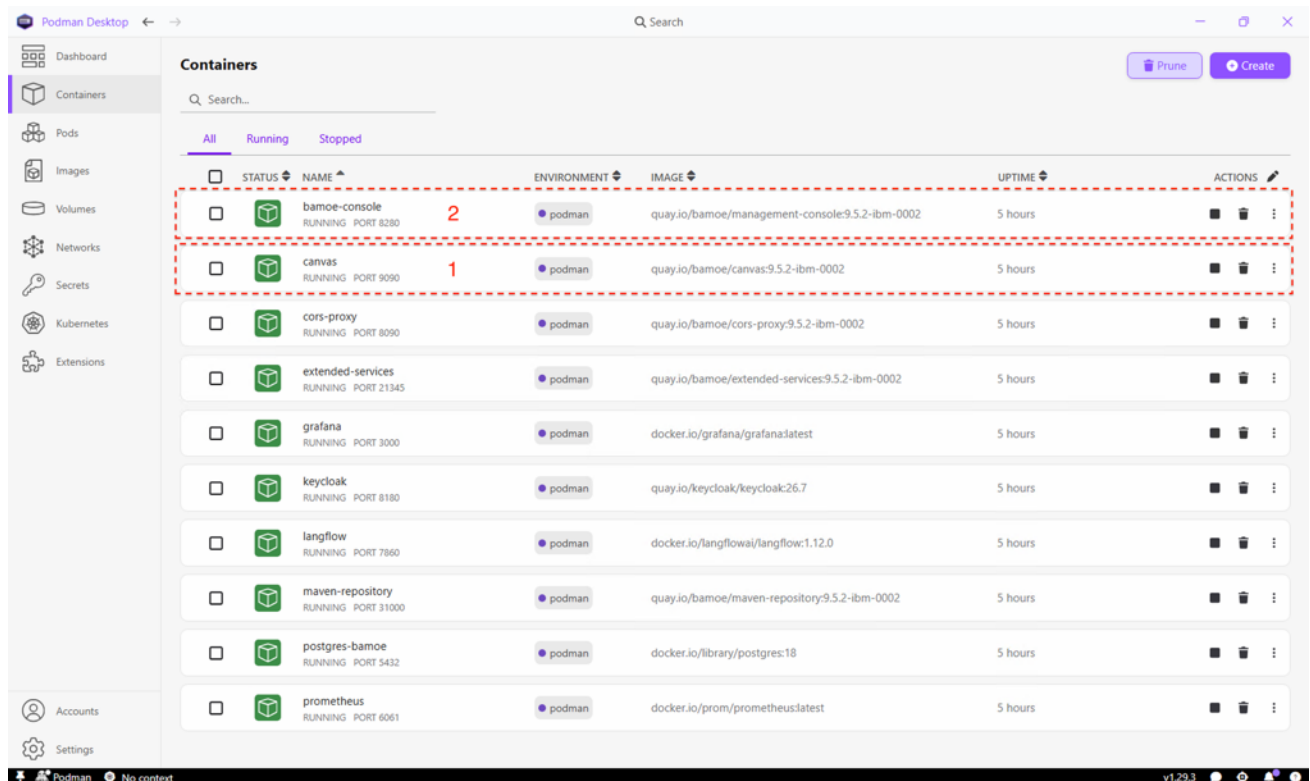
```
User: techzone
Password: IBMDemOs!
```

#### 2.1.1 About the Environment:

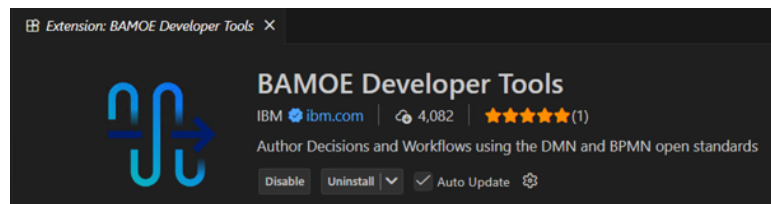
We built this environment so that you can practically use IBM Business Automation Manager Open Editions, so all applications are installed locally. To perform this lab, it will not be necessary to use any external services. All applications are running in Podman containers; you can view them by accessing Podman Desktop.

For this laboratory we will use the applications:

1. [IBM BAMOE Canvas](#)
2. [IBM BAMOE Management Console](#)



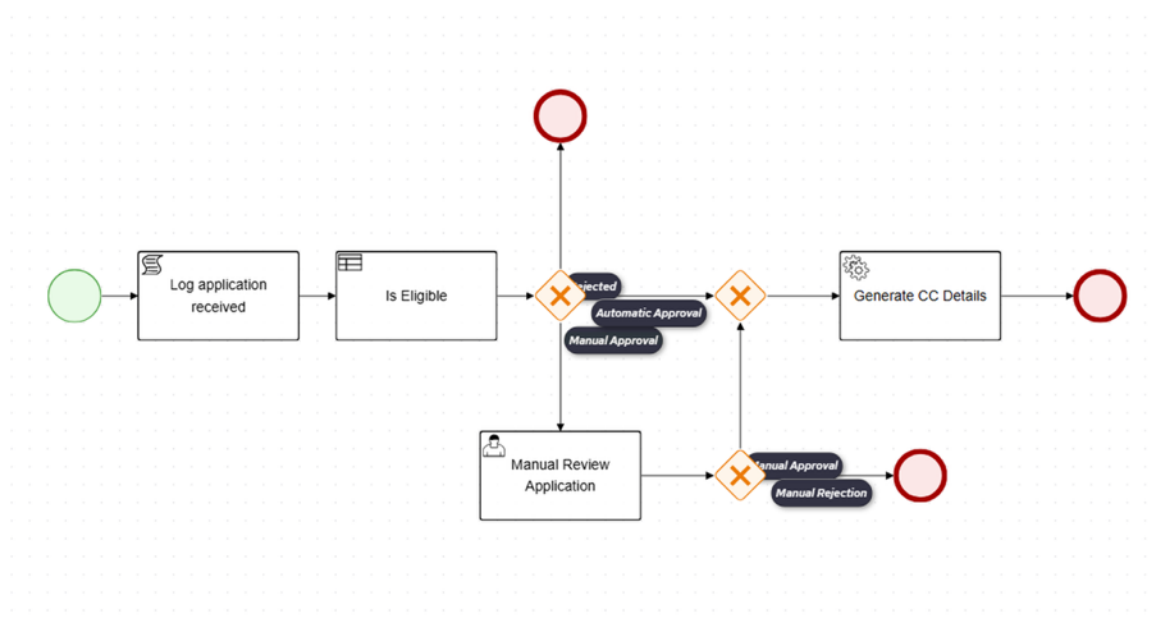
3. For deployment and testing of your applications, VS Code with the [BAMOE Developer Tools](#) extension will be used.



### 3 The use case Lab

Let's work with a fictional **Credit Card Application** process, where the **Workflow** receives the applicant's data and executes a DMN to identify if the application can be:

4. **Automatically approved:** If the applicant's data meets the requirements, the card will be generated automatically.
5. **Automatically rejected:** If the applicant's data does not meet the minimum requirements, the application will be automatically rejected.
6. **Manually approved:** There is a scenario where the application analysis needs to be done manually by a user, who can also approve or reject the application.



Therefore, this lab was divided into 4 exercises, where:

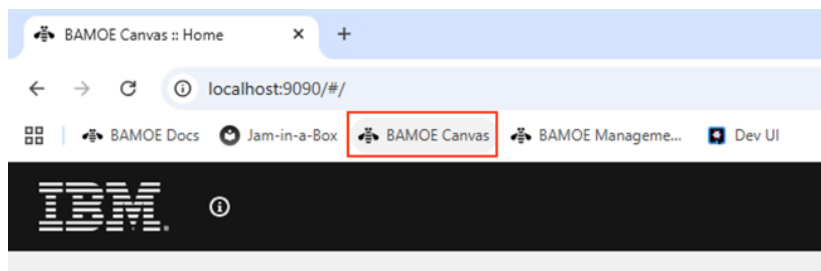
- Exercise 1: Create a DMN using BAMOE Canvas
- Exercise 2: Create a workflow using BAMOE Canvas
- Exercise 3: Run your project using VS Code
- Exercise 4: Connect your project to the BAMOE Management Console

Next, the first exercise begins. Happy studying!

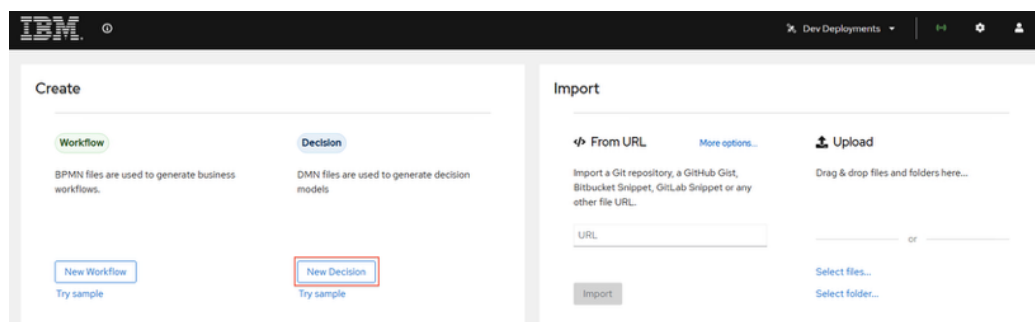
## 4 Exercise 1: Create a DMN using BAMOE Canvas

In this use case, you will learn how to interact with Java classes within a workflow. Therefore, clone this GitHub repository containing the necessary classes to execute the Credit Card Approval process.

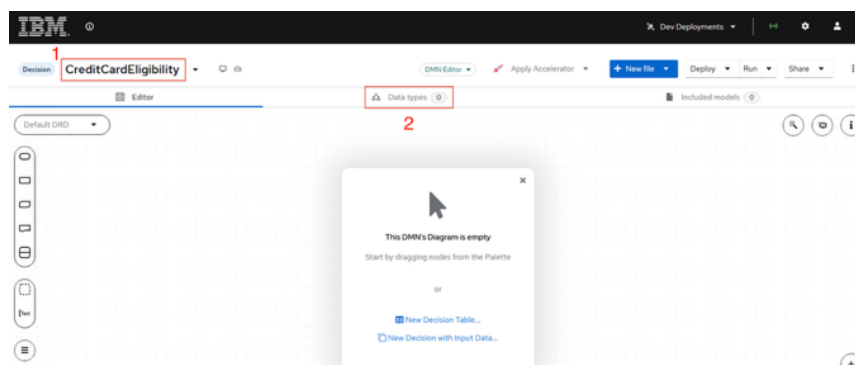
- Open the **Chrome browser** and access the **BAMOE Canvas** (saved in your bookmarks bar):



- Let's create a new DMN, so click on **"New Decision"**.



- I renamed the DMN file to **"CreditCardEligibility"**.
- Click on the **"Data Types"** tab to create the DMN inputs and outputs variables.



- Click the button **Create a custom data type**
- In this model, we will have 2 data types:



- **“Applicant”**: Contains all the person's data to later apply the eligibility rule. We will then receive this data from the Workflow.

- Enter the name **“Applicant”**, select **“is struct?”**:

The screenshot shows the 'CreditCardEligibility' decision editor. In the left sidebar, 'Applicant' is selected. The main area shows the configuration for 'Applicant'. The name 'Applicant' is entered in the top field (labeled 1). Below it, the 'Is struct?' toggle is turned on (labeled 2). The 'Is collection?' toggle is turned off.

- Click the **+** to add the **properties** and select the **type**:

Properties in 'Applicant' **+** 3

| Name           | Is struct?               | Type    | Is collection?           |
|----------------|--------------------------|---------|--------------------------|
| 4 student      | <input type="checkbox"/> | boolean | <input type="checkbox"/> |
| 5 annualIncome | <input type="checkbox"/> | number  | <input type="checkbox"/> |
| 6 creditScore  | <input type="checkbox"/> | number  | <input type="checkbox"/> |
| 7 age          | <input type="checkbox"/> | number  | <input type="checkbox"/> |

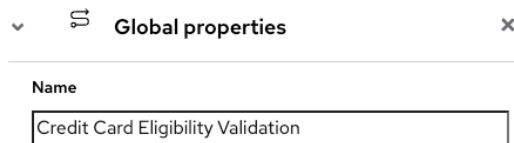
- **“EligibilityResult”**: This will be the DMN's return to the Workflow, that is, the result of the rule.

- Click the **+** located in the left corner, name it **“EligibilityResult”**, and select the ***String type***.

The screenshot shows the 'EligibilityResult' data type configuration. In the left sidebar, 'EligibilityResult string' is selected (labeled 1). The main area shows the configuration for 'EligibilityResult'. The name 'EligibilityResult' is entered in the top field (labeled 2). Below it, the 'Type' dropdown is set to 'string' (labeled 3). The 'Is collection?' and 'Is struct?' toggles are both turned off. The 'Constraints' section shows 'None' selected.

- Click on the **Editor** tab to begin modeling the DMN.
- In the left corner we have the elements that make up the DMN notation.

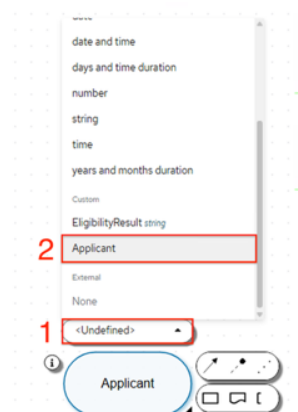
- Click the white box to clear any selection, and if the properties panel is not open, on your right-hand side, click the button  to access the workflow properties. Under **"Global Properties"**, change the DMN name to **"Credit Card Eligibility Validation"**.



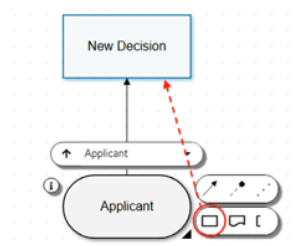
- we need to add the first **input node**; simply click and drag the **"Input Data"** to the whiteboard. I renamed it to "Applicant".



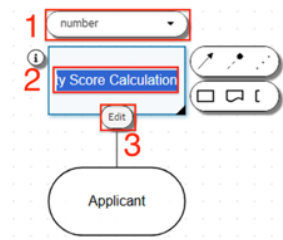
- In the top field of the input, select the **"Applicant"** Data Type that we created.



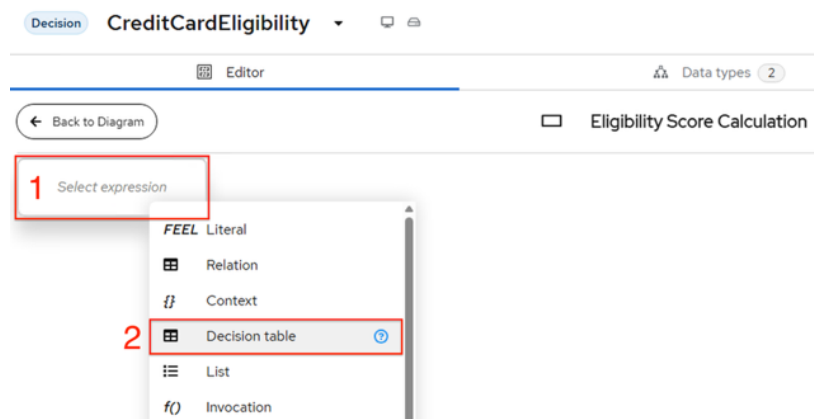
- With the input selected, let's create the first rule element. Select the rectangle and position it on the whiteboard.



- Select **“number”**, click the text to rename it to **“Eligibility Score Calculation”**, and then click “Edit” to create the rule.



- Click on **“Select expression”** to choose the type of rule that will be created in this node.
- Select **“Decision table”**.



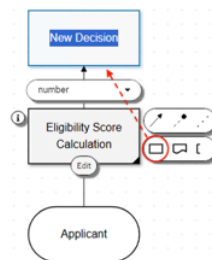
- Fill in your decision table with the following scenarios:

← Back to Diagram Eligibility Score Calculation

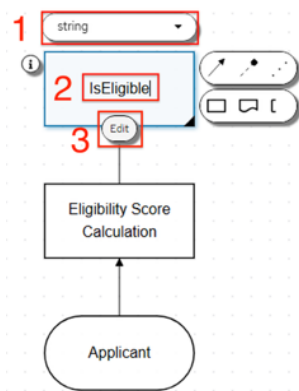
| U | Applicant.student<br>(boolean) | Applicant.annualIncome<br>(number) | Applicant.creditscore<br>(number) | Applicant.age<br>(number) | Eligibility Score...<br>(<Undefined>) | Annotations              |
|---|--------------------------------|------------------------------------|-----------------------------------|---------------------------|---------------------------------------|--------------------------|
| 1 | -                              | -                                  | -                                 | < 18                      | 0                                     | // Your annotations here |
| 2 | -                              | -                                  | < 550                             | >= 18                     | 20                                    |                          |
| 3 | false                          | < 20000                            | [550..700]                        | >= 18                     | 40                                    |                          |
| 4 | false                          | >= 20000                           | [550..700]                        | >= 18                     | 60                                    |                          |
| 5 | false                          | -                                  | >= 700                            | >= 18                     | 80                                    |                          |
| 6 | true                           | -                                  | >= 550                            | >= 18                     | 70                                    |                          |

To create a new line, position the cursor below the existing line

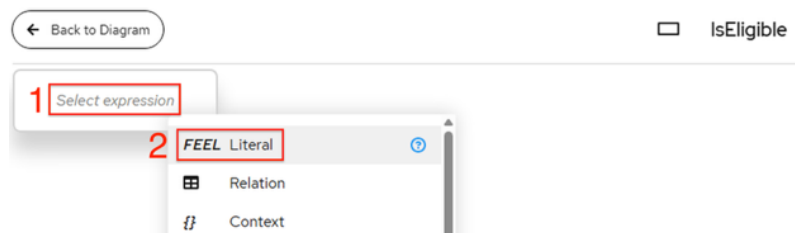
- Click the button to return to the diagram
- Let's create another node to create the final decision:



- Now select the output type as **"string"**, rename it to **"IsEligible"**, and click edit to configure the rule.

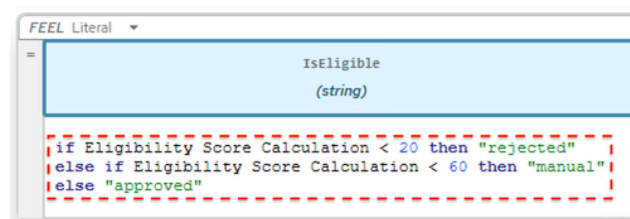


- Here we will select the expression FEEL.

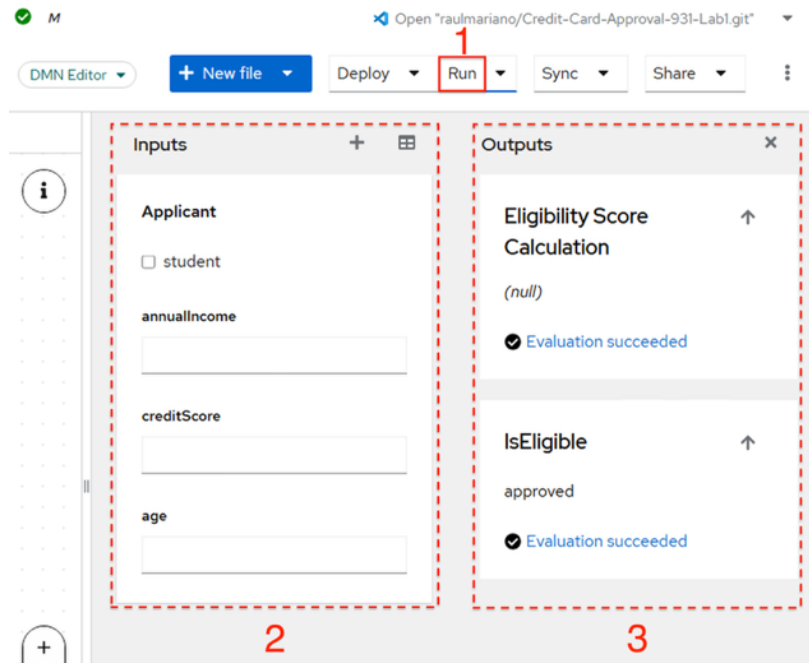


- Enter the code to validate final eligibility:

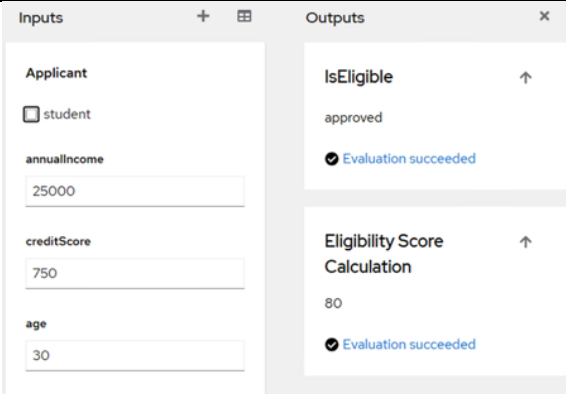
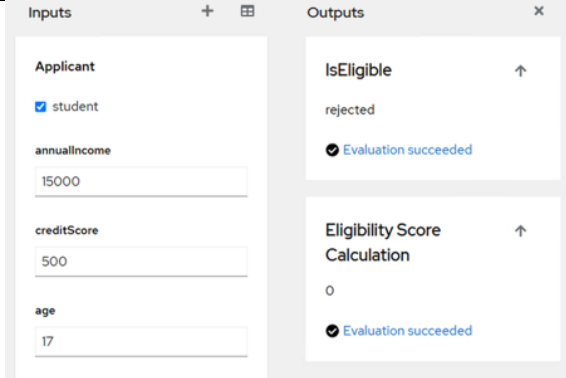
```
if Eligibility Score Calculation < 20 then "rejected"
else if Eligibility Score Calculation < 60 then "manual"
else "approved"
```



- Go back to the diagram to test your DMN. Click on .
- BAMOE Canvas allows you to test your DMN during modeling. In the upper right bar, click "Run" to access the form (inputs) and the rule result in outputs.

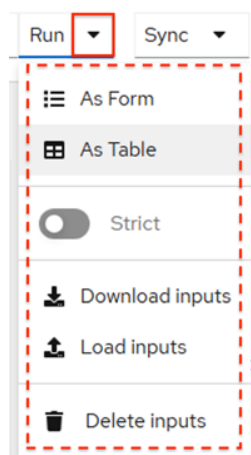


- Use the following data to test the 3 possible scenarios:

| SCENARIO                   | Inputs/Outputs                                                                       |
|----------------------------|--------------------------------------------------------------------------------------|
| <i>Automatic approval</i>  |  |
| <i>Automatic rejection</i> |  |

|                             |  |                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><i>Manual Review</i></p> |  | <div>Inputs</div> <div> <div>Applicant</div> <div> <input checked="" type="checkbox"/> student         </div> <div>           annualIncome<br/> <input type="text" value="15000"/> </div> <div>           creditScore<br/> <input type="text" value="500"/> </div> <div>           age<br/> <input type="text" value="30"/> </div> </div> | <div>Outputs</div> <div> <div>IsEligible</div> <div>↑</div> <div>manual</div> <div> <input checked="" type="checkbox"/> Evaluation succeeded         </div> </div> <div> <div>Eligibility Score Calculation</div> <div>↑</div> <div>20</div> <div> <input checked="" type="checkbox"/> Evaluation succeeded         </div> </div> |
|-----------------------------|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- Explore the DMN editor further; for example, this button gives you access to other features for testing.

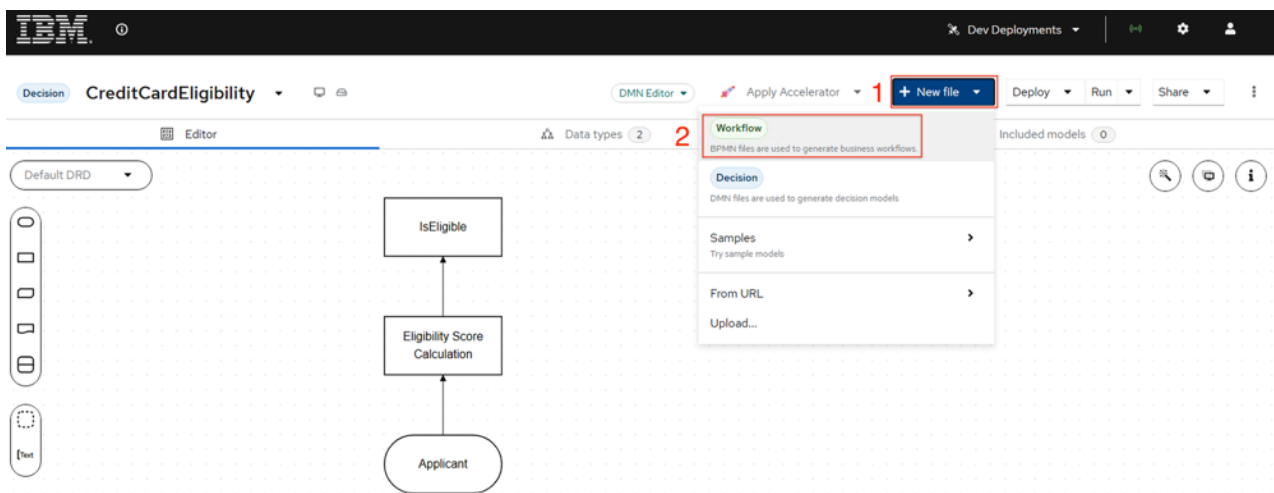


Great, you've finished the first exercise in this lab. Next, we'll build the Workflow.

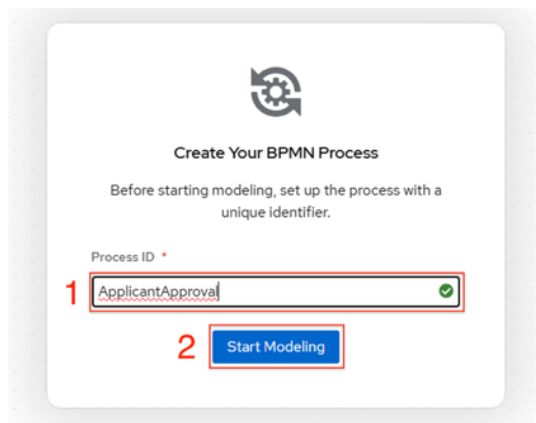
## 5 Exercise 2: Create a workflow using BAMOE Canvas

Now that your project has been created, and you've already created your first automated rules (DMN), let's build the Workflow, which will be responsible for automatically directing requests to their destination.

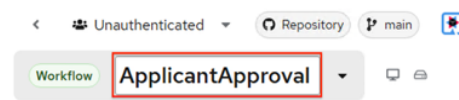
- Back to Canvas, where we left off in the previous exercise.
- In the upper right menu of the DMN editor, click the "New file" button, then select "Workflow".



- Enter a name of your choice for the **"Process ID"**.



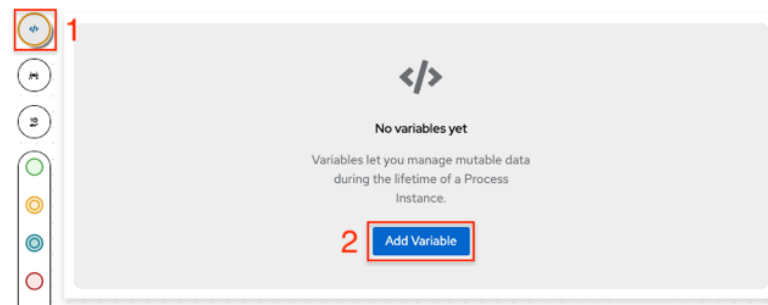
- Change the name of your workflow to **"ApplicantApproval"**.



- Before we start designing the workflow, let's register the "Process Variables". These are the variables through which data will be received and how the automation

returns the result; in other words, we'll create the inputs and outputs for our BAMOE application.

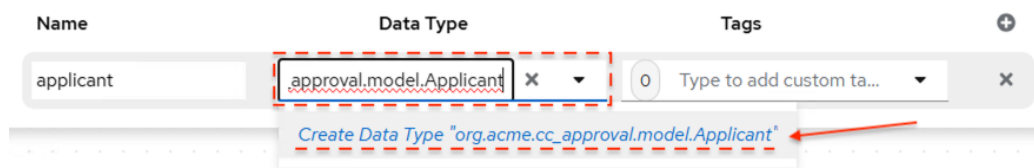
- Click the first button on the toolbar, then the **"Add Variable"**.



- Now click on  for all 3 variables:



- If you aren't very familiar with Java, note that we will configure two variables **"applicant"** and **"creditCard"**, whose data types are Java class paths. And to include them:
  - Write the path exactly as shown in the table below.
  - Now click the button to register the new data type.



- Finally, add the variables with their respective values:

| Name       | Data Type                             | Tags   |
|------------|---------------------------------------|--------|
| approval   | String                                | Output |
| applicant  | org.acme.cc_approval.model.Applicant  | Input  |
| creditCard | org.acme.cc_approval.model.CreditCard | Output |



- Click the white box to clear any selection, and if the properties panel is not open, click  to access the workflow properties. Under **"Process"**, change the Workflow name to **"approval"**.

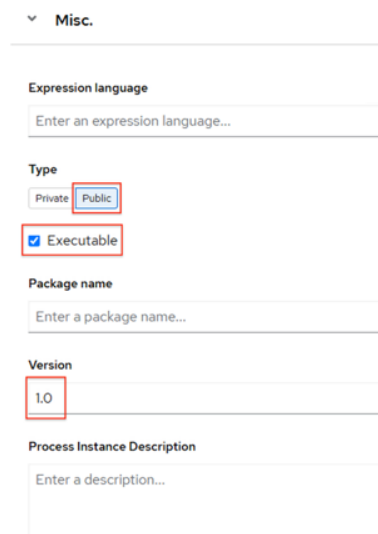


Process

Name

approval

- Scroll down and access the **"Misc."** section. Make sure yours matches the image:



Misc.

Expression language

Enter an expression language...

Type

Private Public

☒ Executable

Package name

Enter a package name...

Version

1.0

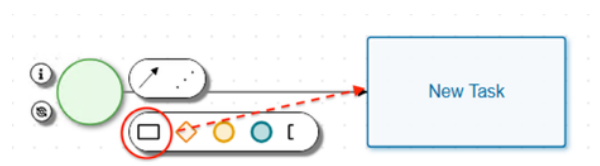
Process Instance Description

Enter a description...

- Alright, let's start the "drag and drop" process and create our workflow.
- Drag the **"Start Event"** onto the whiteboard.

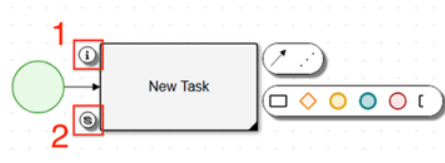


- To model, I like to select the previous step, choose and drag the next step, so the steps are already connected. But nothing prevents you from choosing the element in the tools in the left column, just like we did with the Start Event.
- Create a new task.

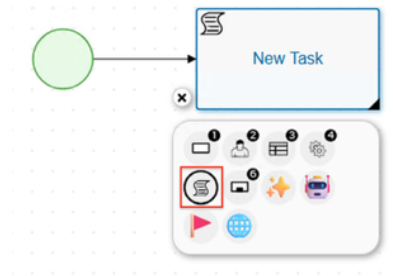


- You'll notice that all the elements will have two icons that we'll use a lot:

- Properties: Here you have access to the element's properties panel; all task settings are made here.
- Type: Here you define the element's type; each element will have its own.

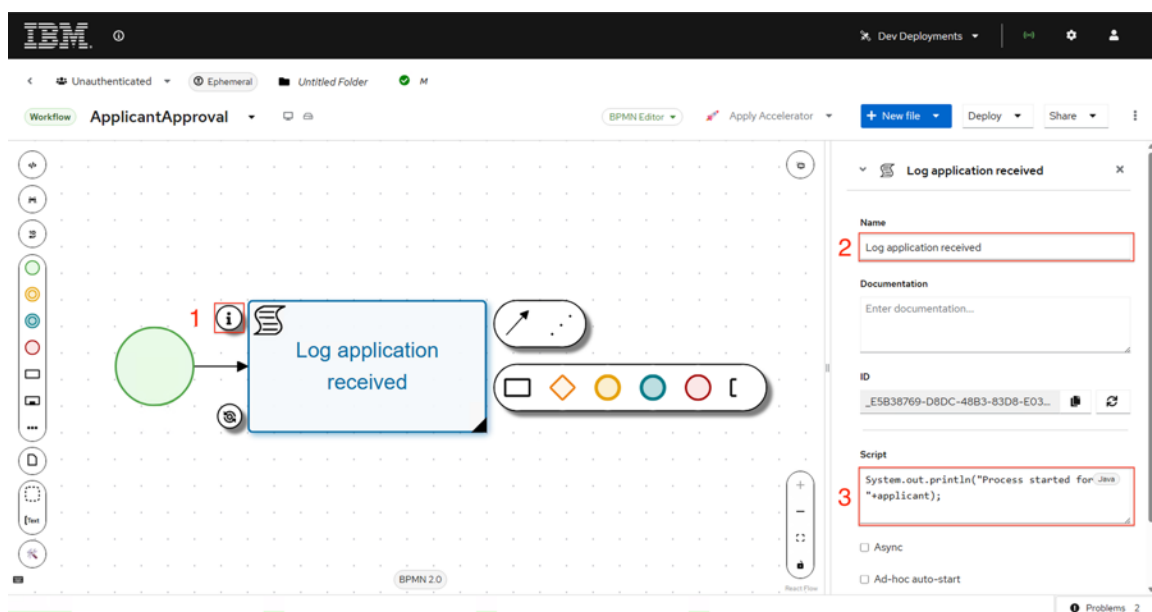


- For the first task, we will select the **"script"** type.

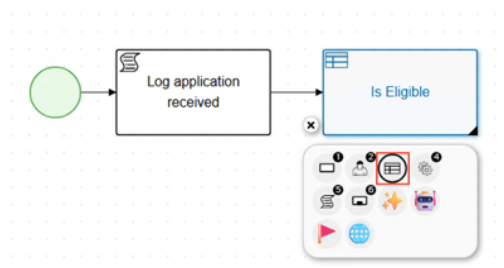


- Access the properties of this task, change the name to **"Log application received"**, and we'll include some Java code to log that the process has started:

```
System.out.println("Process started for: "+applicant);
```



- The next task will be called **"Is Eligible"**, follow the same procedure explained previously to drag it. It will be of the **"Business Rule Task"** type.



- Navigate through the properties panel, and let's call the DMN we created. Select **"DMN"** button, then select **"CreditCardEligibility.dmn"**.

**Name**  
Is Eligible

**Documentation**  
Enter documentation...

**ID**  
\_EE9B6C8C-751F-4393-9582-448D7D5598...

**Implementation**  
Choose between Rules with DRL and Decisions with DMN.

☐ DRL ☒ DMN

DMN model relative path  
CreditCardEligibility.dmn

Enter a relative path...

- Check if the fields have been filled in.

**Implementation**  
Choose between Rules with DRL and Decisions with DMN.

☐ DRL ☒ DMN

DMN model relative path  
CreditCardEligibility.dmn

DMN model namespace  
https://kie.org/dmn/\_E681D495-529E-4A3F-B0A5-...

DMN model name  
Credit Card Eligibility Validation

- A little further down, we'll map the input and output data. Click on **"Data mapping"**.

↔ **Data mapping** ✎

- Click the  and  buttons to add.
- Look, here we're going to send (input) the **"Applicant"** data to the DMN. Note that the Data Type is a global variable that we configured at the beginning of this exercise. And our Workflow will receive from the DMN (output) the result of the rule we created, of type String.
- Click on **"Save"**.

**Data mapping for 'Is Eligible'**

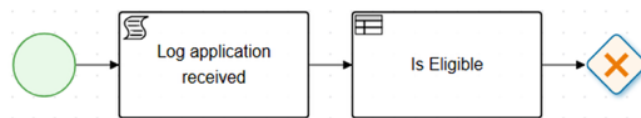
| Inputs    |                         |                 |
|-----------|-------------------------|-----------------|
| Name      | Data Type               | Source          |
| Applicant | org.acme.cc_approv... X | Var applicant X |

| Outputs    |           |                |
|------------|-----------|----------------|
| Name       | Data Type | Target         |
| IsEligible | String X  | Var approval X |

**Save**

- Now that we have the result of the rule, the next step will be to add a Gateway .



- Let's take this opportunity to create the 3 possible paths:
  - Case 1:**
    - If the card request is **automatically rejected**, then the process will be terminated.
    - So, let's add a **"End Event"**. To configure this rule, select **"sequence flow"**, and in its properties, enter the Java code to perform this validation:

```
return approval.toLowerCase().equals("rejected");
```

**Sequence Flow**

Name: Rejected **2**

Documentation: Enter documentation...

ID: \_75580860-C67B-45BF-9DFA-A1D86D...

Priority: Enter priority...

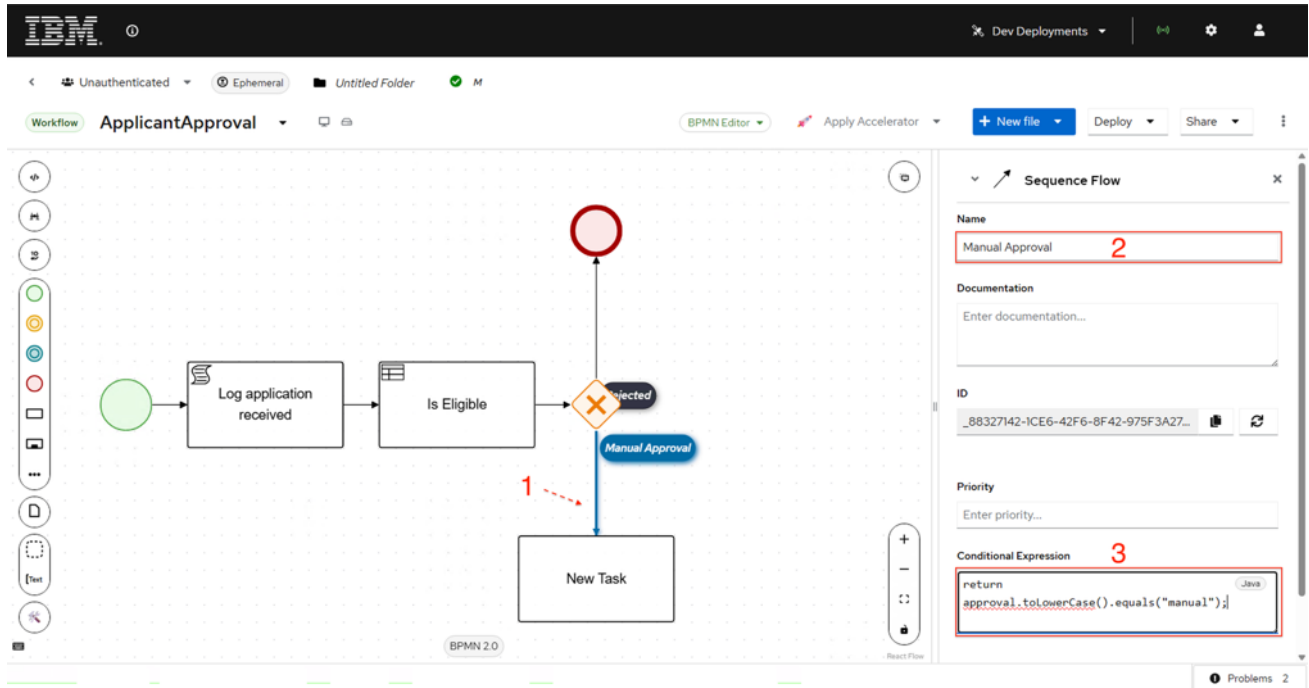
Conditional Expression **3**

```
return approval.toLowerCase().equals("rejected");
```

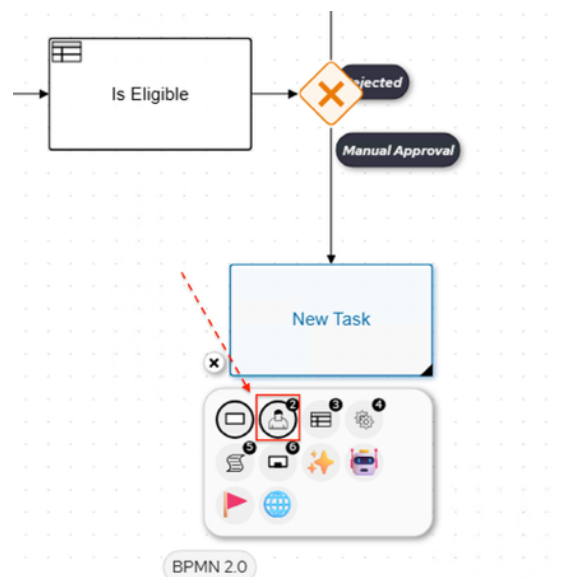
○ **Case 2:**

- If the card request needs to be reviewed by a person, then the next step is to create another task.
- Add a new Task, and similarly select the **"Sequence Flow"** to configure the **"Conditional Expression"** field:

```
return approval.toLowerCase().equals("manual");
```



- Select the new task to configure it. It will be of the **"Human Task"** type.



- In the properties, fill in the fields: **"Name"**, **"Task Name"**, and **"Actors"**.

| Name                      | Task Name    | Actors |
|---------------------------|--------------|--------|
| Manual Review Application | manualReview | jdoe   |

- In the **"Data Mapping"** section, we will configure the data that will be displayed to the user, and what the result of their decision will be.

Data mapping for 'Manual Review Application'

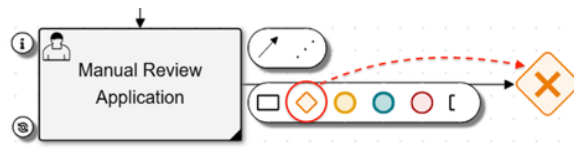
| Inputs    |                       |               |
|-----------|-----------------------|---------------|
| Name      | Data Type             | Source        |
| Applicant | org.acme.cc_approv... | Var applicant |

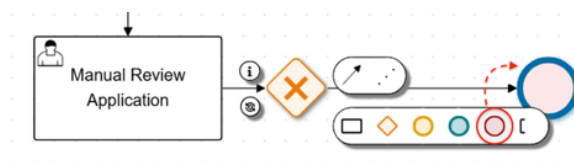
| Outputs  |           |              |
|----------|-----------|--------------|
| Name     | Data Type | Target       |
| approval | String    | Var approval |

Save

- Just like we did before, we'll validate whether the user approved or rejected the application. So, for the next step, include a **new gateway**.

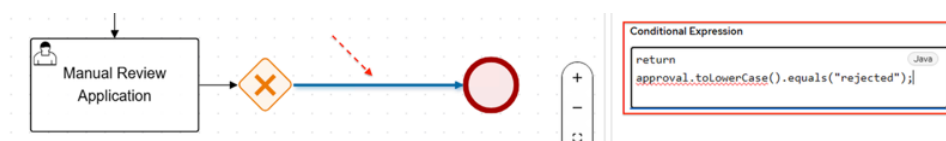


- Let's first draw the path for cases rejected by the user, where the flow will end. Add a "End Event".

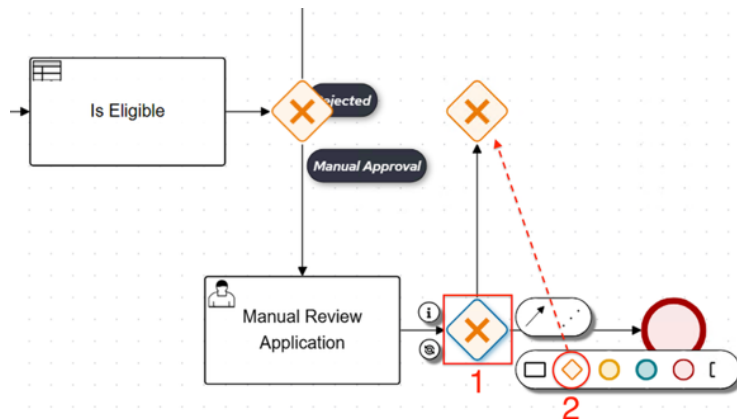


- Take this opportunity to place the rule (Java code) in the **sequence flow** properties.

```
return approval.toLowerCase().equals("rejected");
```

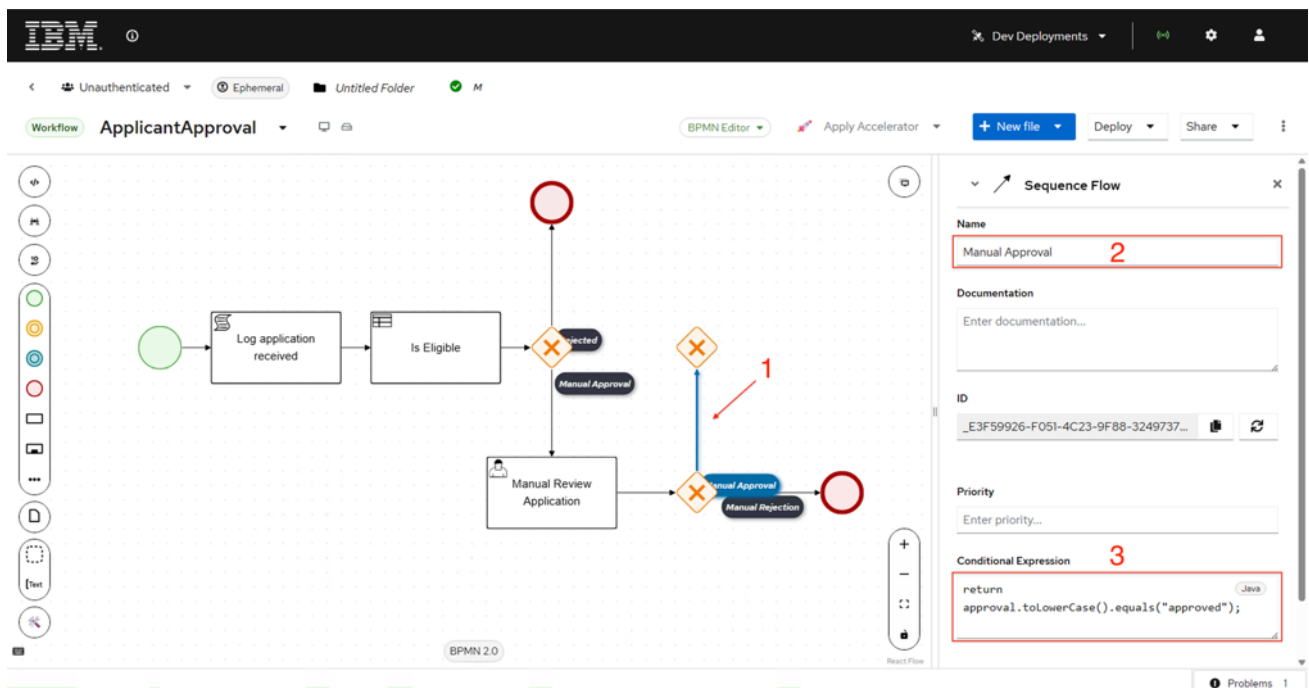


- Now that we've resolved another path, let's return to the "happy path" of this process, that is, if the user approves the card manually, the flow should return to the main flow, to the cases where the card was approved automatically.
- You know we're following the BPMN notation, so add a new gateway to converge the flows, see:



- Finally, select the new sequence flow to configure the rule.

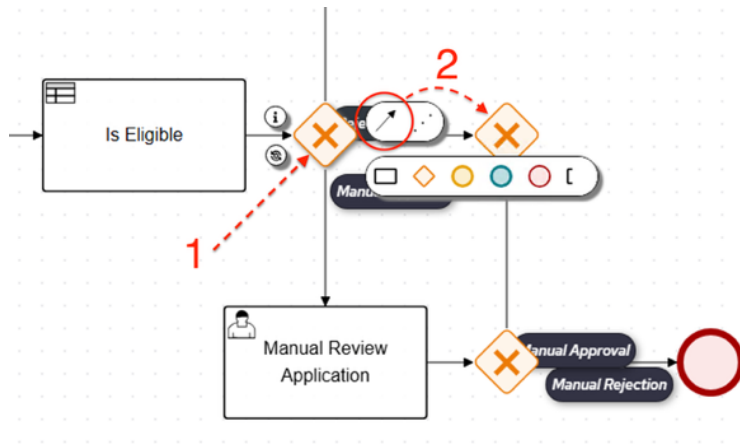
```
return approval.toLowerCase().equals("approved");
```



- **Case 3:**

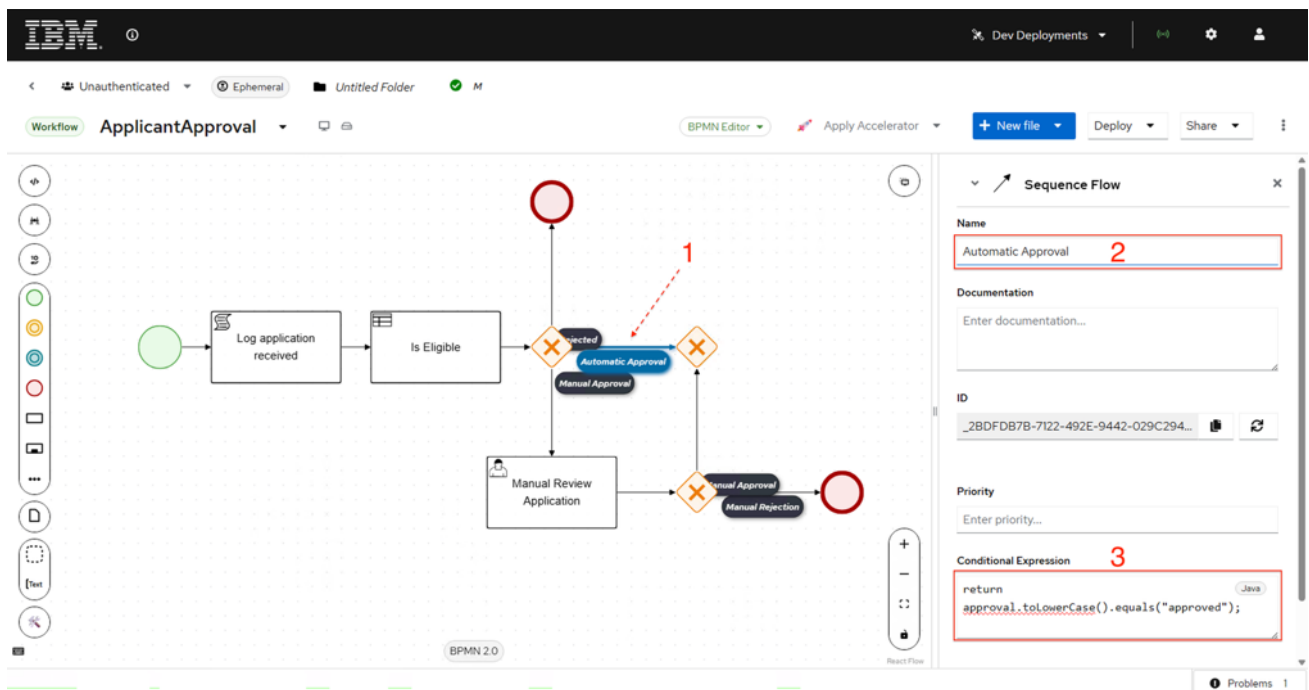
- For the last proposed case, let's configure the "happy path," the scenario in which the card was approved.
- We need to connect 2 gateways now, so select the first one and connect to the next one, see:



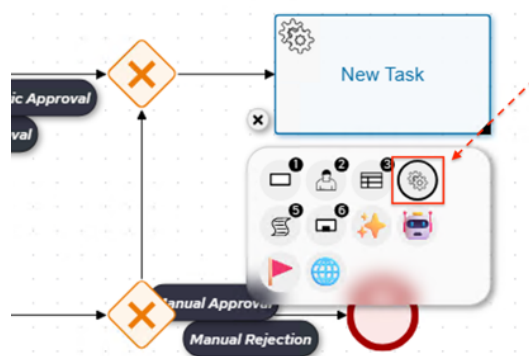


- Select the sequence flow to configure the rule.

```
return approval.toLowerCase().equals("approved");
```



- After merging the paths (gateway that converges the manual and automatic approval flows), we will create a new task of the service type.

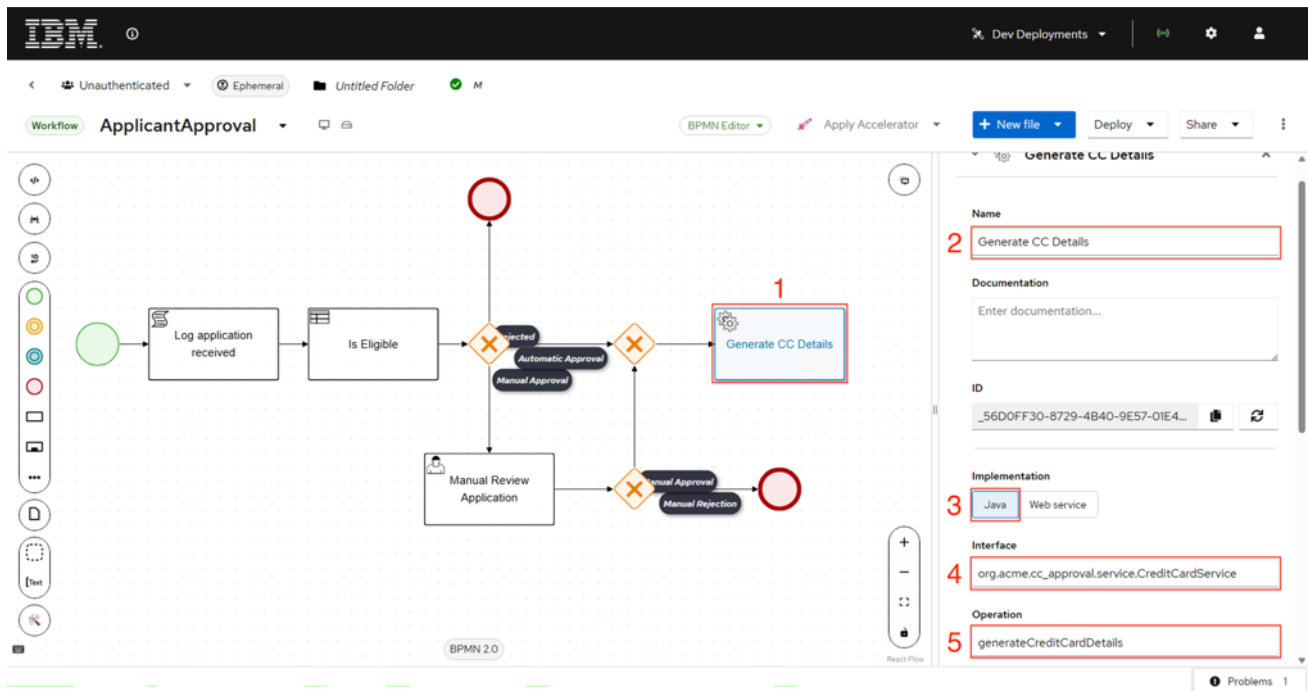


- This task will be responsible for generating the card data. And how will this be done? We have another Java code ready to execute this, so go to the task properties and configure it:

**Name:** Generate CC Details

**Interface:** org.acme.cc\_approval.service.CreditCardService

**Operation:** generateCreditCardDetails



- And just like the other tasks, we need to map the data. So, go to **"Data Mapping"** and specify the input and output variables.

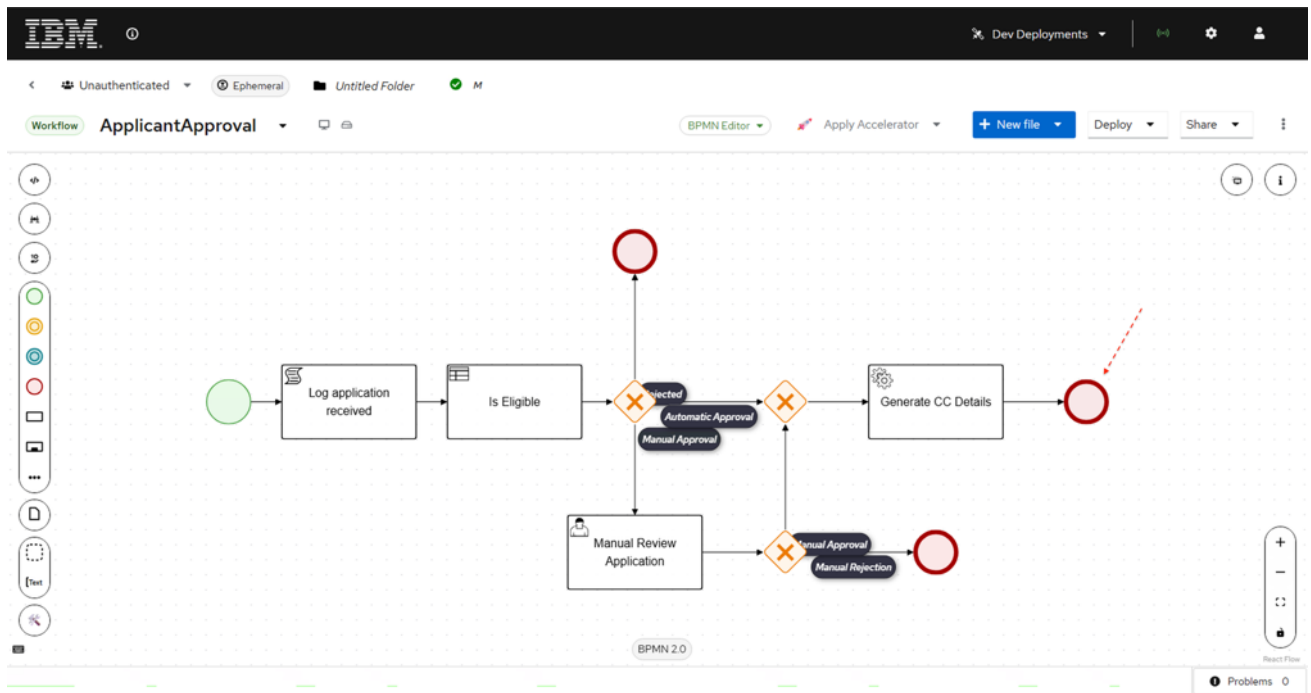
The screenshot shows the 'Data mapping for Generate CC Detail' dialog box. It has two main sections: 'Inputs' and 'Outputs'. Each section contains a table with columns for Name, Data Type, and Source/Target. A 'Save' button is at the bottom left.

| Inputs    |                       |               |
|-----------|-----------------------|---------------|
| Name      | Data Type             | Source        |
| applicant | org.acme.cc_approv... | Var applicant |

| Outputs    |                       |                |
|------------|-----------------------|----------------|
| Name       | Data Type             | Target         |
| creditCard | org.acme.cc_approv... | Var creditCard |

- Now we can add the "End Event".

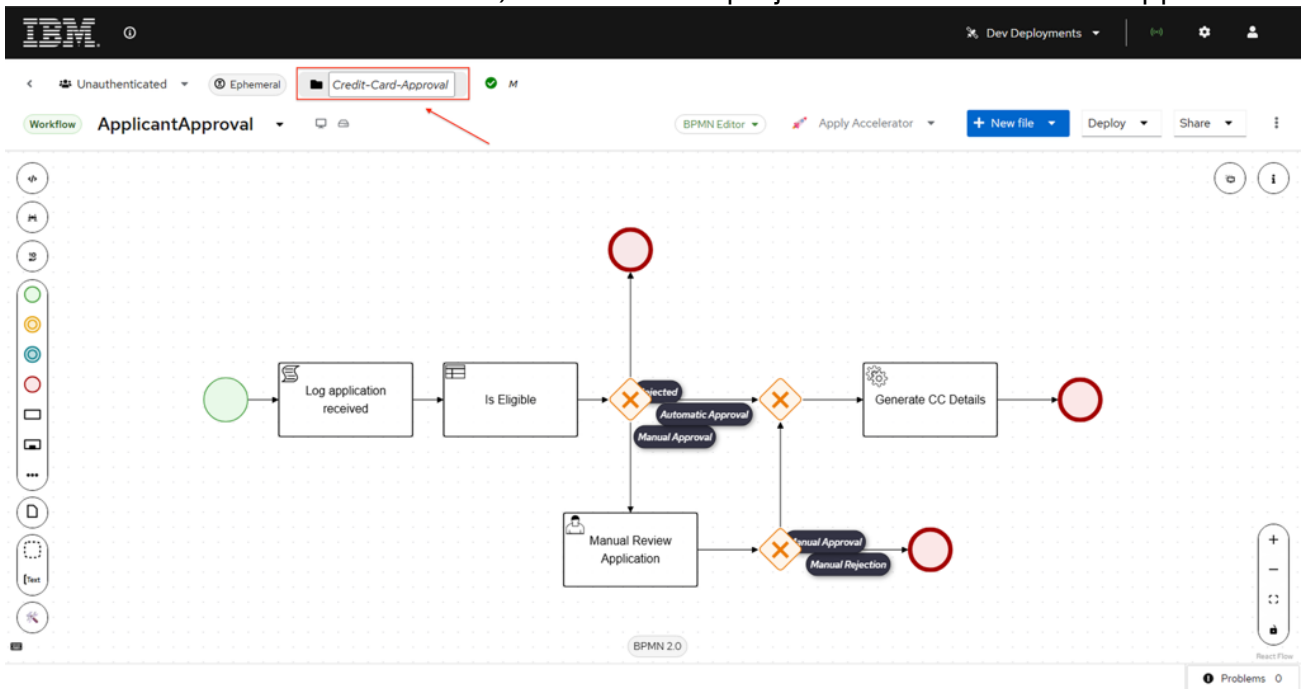


Congratulations, your workflow is ready! This concludes another exercise and finalizes the modelling proposed in this Lab. Next, we will explore the project in VS Code and test its functionality.

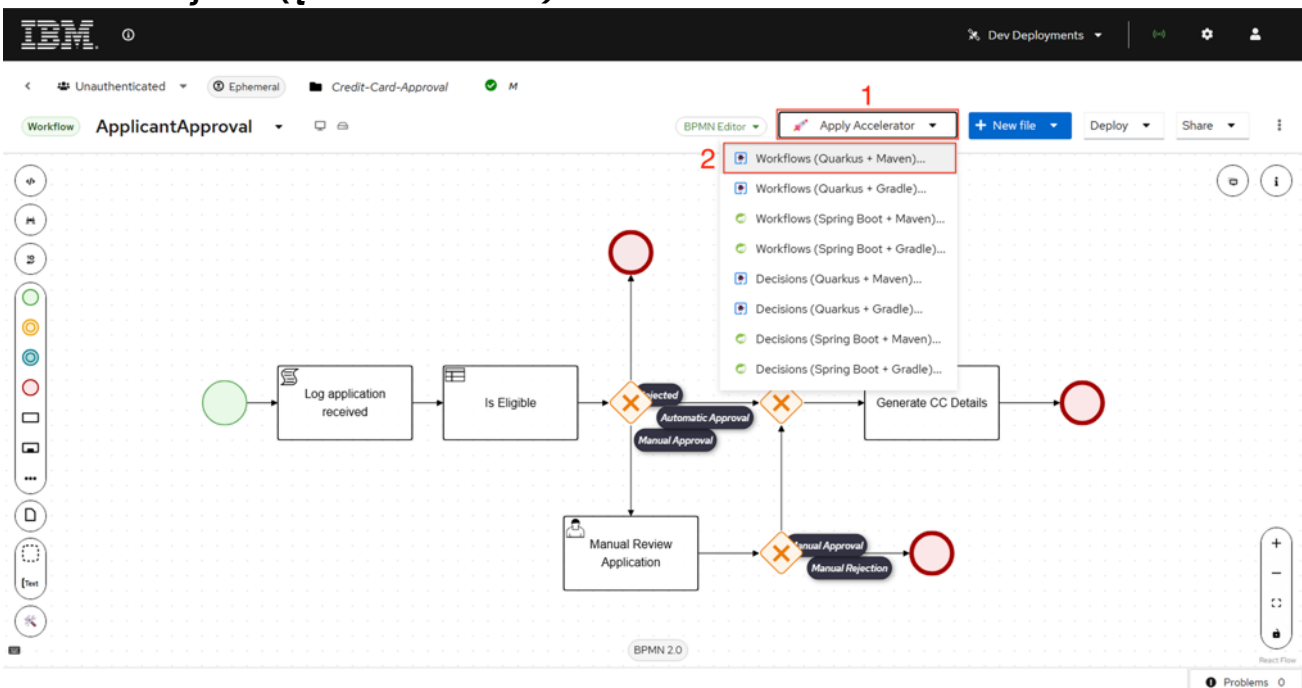
## 6 Exercise 3: Run your project using VS Code

In the previous step, you learned how to model your Workflows and Decisions in BAMOE Canvas. You should also have explored that it's possible to connect to some providers (for example, GitHub), but we can also download the project folder (.zip) locally. So now, let's use this feature to explore this project in more depth using VS Code, as well as test it.

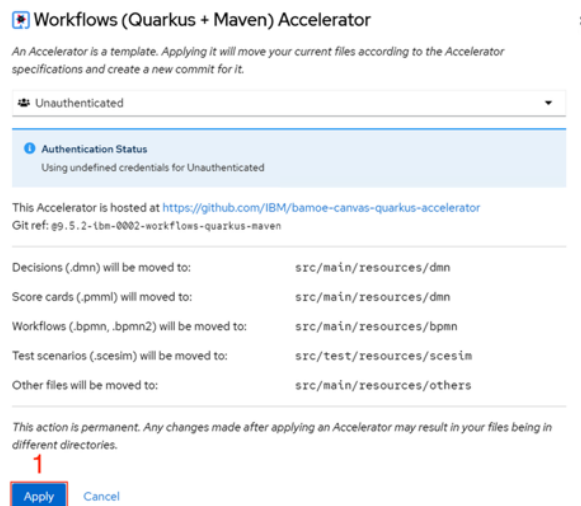
- Back in IBM BAMOE Canvas, let's name the project folder "Credit-Card-Approval".



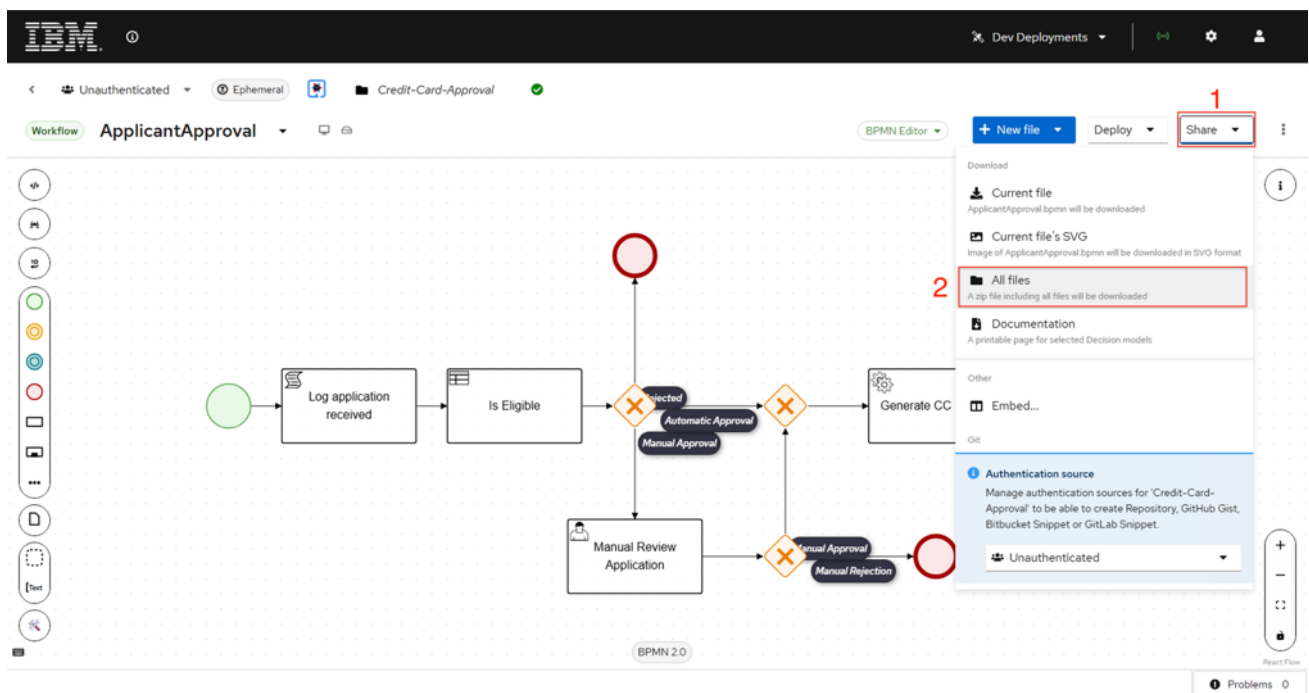
- To simplify a large part of the project setup process, we will **"Apply Accelerator" > "Workflows (Quarkus + Maven)..."**



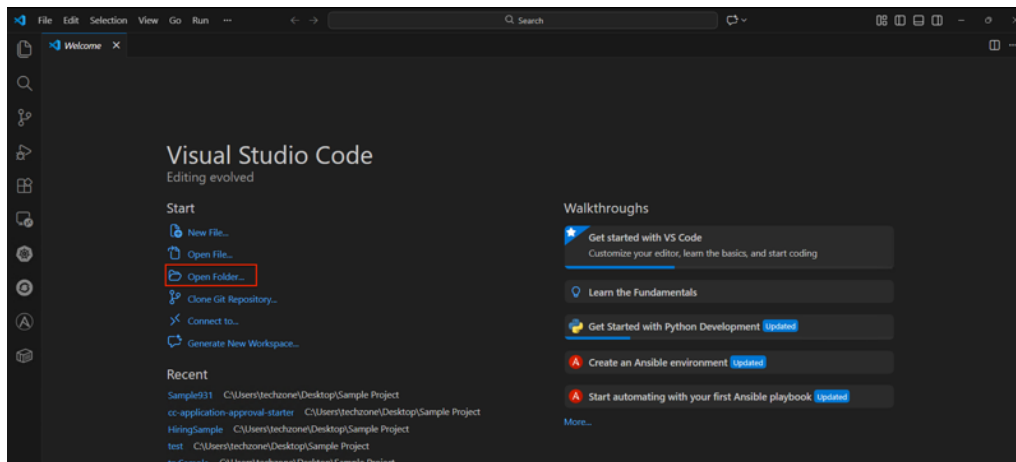
- Click **"Apply"**.



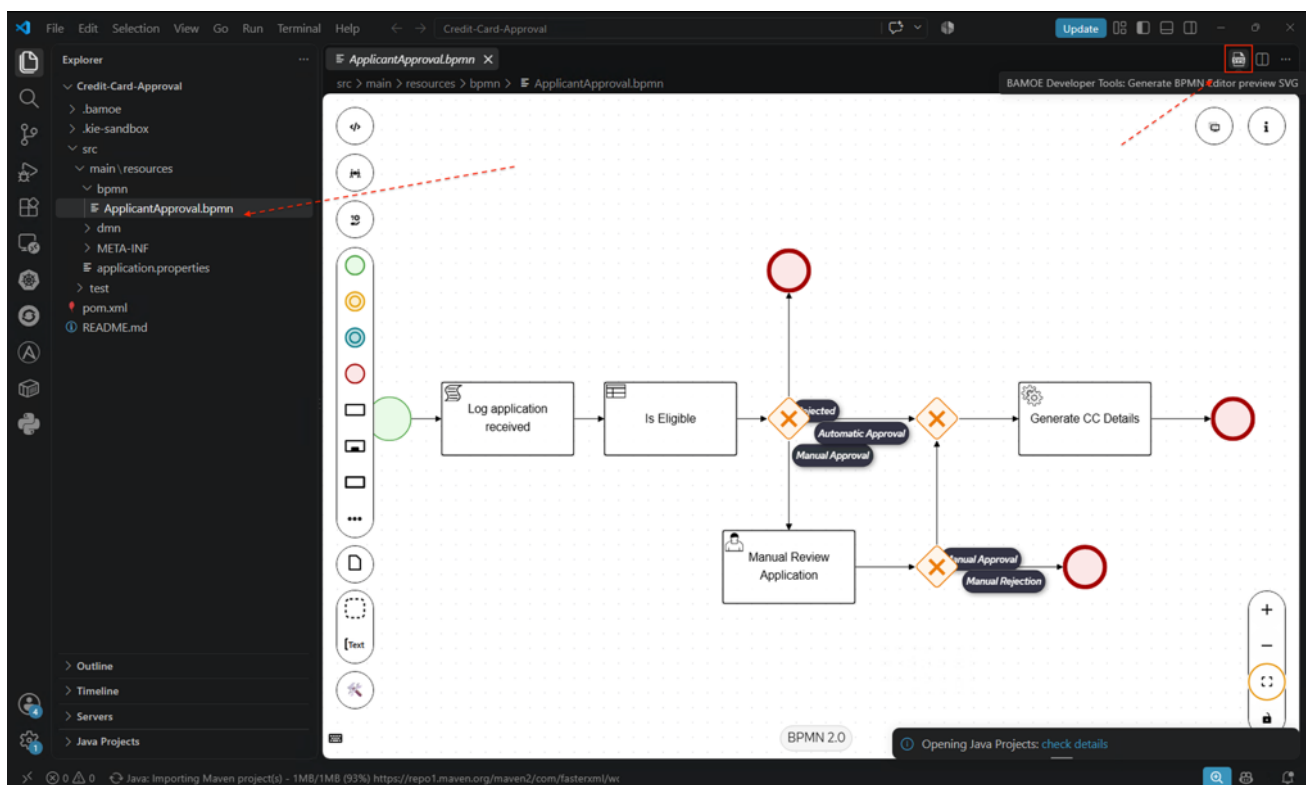
- In the upper right corner, click **"Share" > "All Files"** and save it to a folder on your computer.



- Open VS Code , pinned to the taskbar.
- Open the project folder in VS Code and explore the project structure and see what features have already been created in this use case.



- Navigate to the **"Credit-Card-Approval\src\main\resources\bpmn"** folder, select the **"ApplicantApproval.bpmn"** file, and click the **"SVG"** button to generate the workflow image. We will use this image to visually monitor the instances.

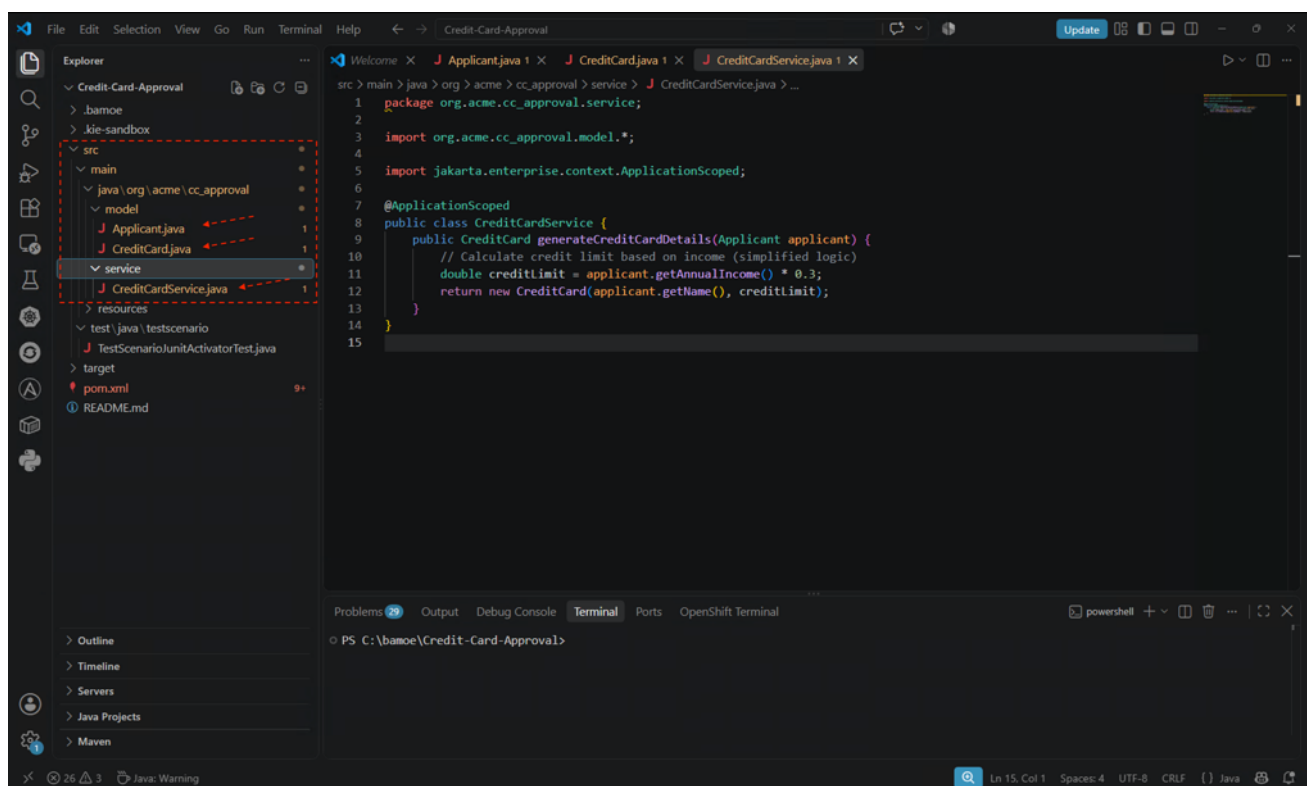


In the previous exercise, you likely noticed that we mapped data originating from Java classes. For example, requester data or approved credit card data is obtained via Java code. Therefore, in the next steps, you need to create three new classes in your project hosted on GitHub. To do this, follow these steps:

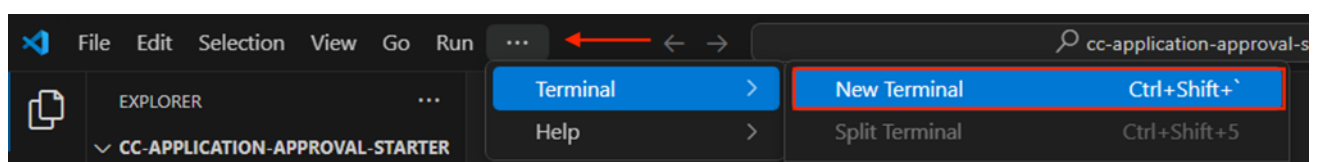
- You can download the class (\*.java) and paste it into the appropriate path, or copy and paste the code via VS Code:

| Jaca Class             | Path                                                              | Download               |
|------------------------|-------------------------------------------------------------------|------------------------|
| Applicant.java         | src/main/java/org/acme/cc_approval/model/Applicant.java           | <a href="#">Github</a> |
| CreditCard.java        | src/main/java/org/acme/cc_approval/model/CreditCard.java          | <a href="#">Github</a> |
| CreditCardService.java | src/main/java/org/acme/cc_approval/service/CreditCardService.java | <a href="#">Github</a> |

- Here is how your project should look after creating them:



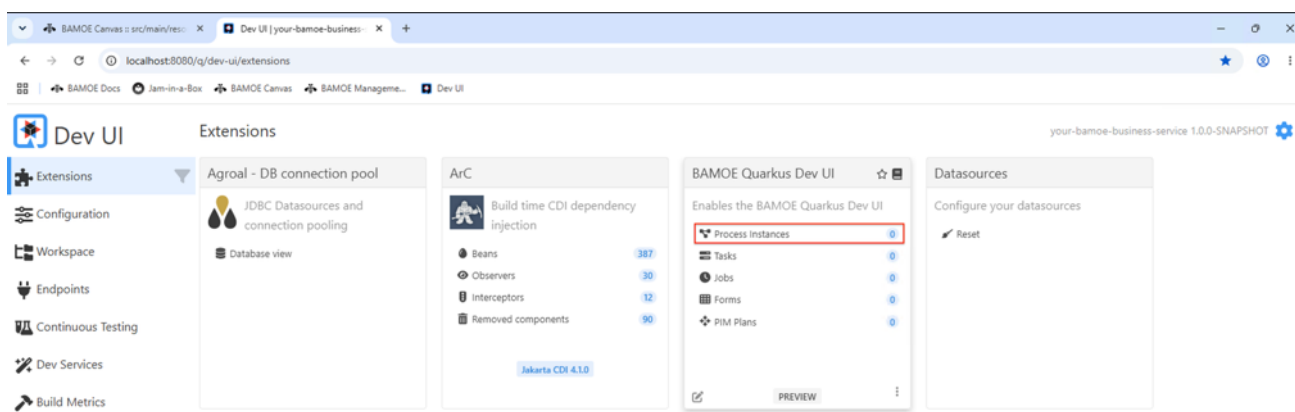
- Let's run the application and test our workflow.
- Open a new terminal, **"Terminal", "New Terminal"**.



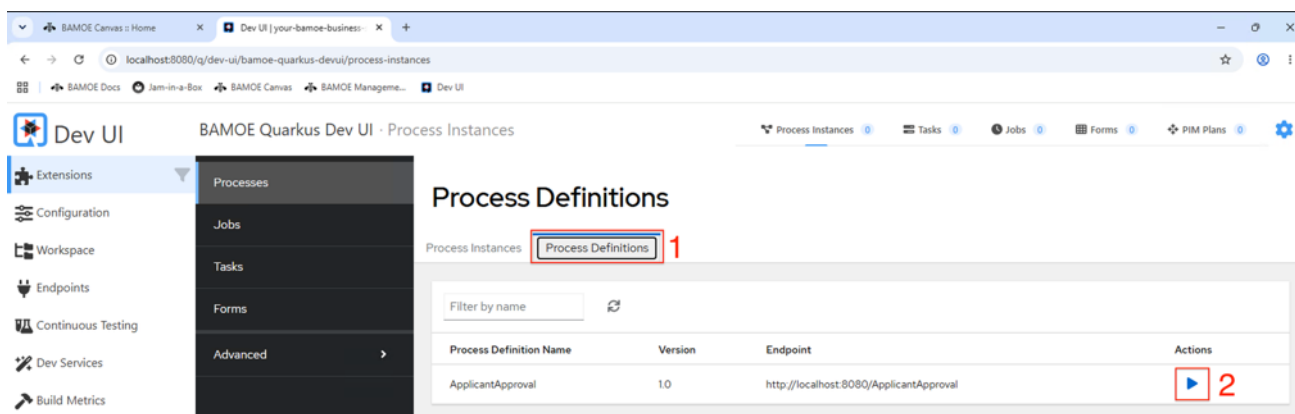
- Execute this command to deploy to development:  
`mvn clean quarkus:dev -Pdevelopment`
- Wait for the process to start. You should see the message Listening on: <http://localhost:8080>. This may take a few minutes.

```
PS C:\Users\techzone\Desktop\Credit-Card-Approval-931> mvn clean quarkus:dev -Pdevelopment
) started in 39.719s. Listening on: http://localhost:8080
2026-02-16 09:39:49,473 INFO [io.quarkus] (Quarkus Main Thread) Profile dev activated. Live Coding activated.
2026-02-16 09:39:49,474 INFO [io.quarkus] (Quarkus Main Thread) Installed features: [agroal, bamoe-ai-agent-task-work-item-handler, bamoe-pi
m, bamoe-quarkus-devui, cdi, flyway, hibernate-orm, jbpmm-addon-usertask-storage-jpa, jdbc-h2, jdbc-postgresql, kie-addon-persistence-jdbc-ext
ension, kie-addon-process-management-extension, kie-addon-process-svg-extension, kie-addon-source-files-extension, kie-flyway, kogito-addons-
quarkus-data-index-jpa, kogito-addons-quarkus-data-index-persistence-jpa, kogito-decisions, kogito-predictions, kogito-processes, kogito-rule
s, messaging, narayana-jta, oidc, reactive-routes, resteasy, resteasy-client, resteasy-jackson, resteasy-multipart, security, security-proper
ties-file, servlet, smallrye-context-propagation, smallrye-fault-tolerance, smallrye-health, smallrye-openapi, smallrye-reactive-messaging-ht
tp, swagger-ui, vertx, vertx-graphql]
```

- Open Google Chrome, and go to <http://localhost:8080/q/dev-ui>.
- Let's understand more about the use case by accessing the **"Process instances"**.

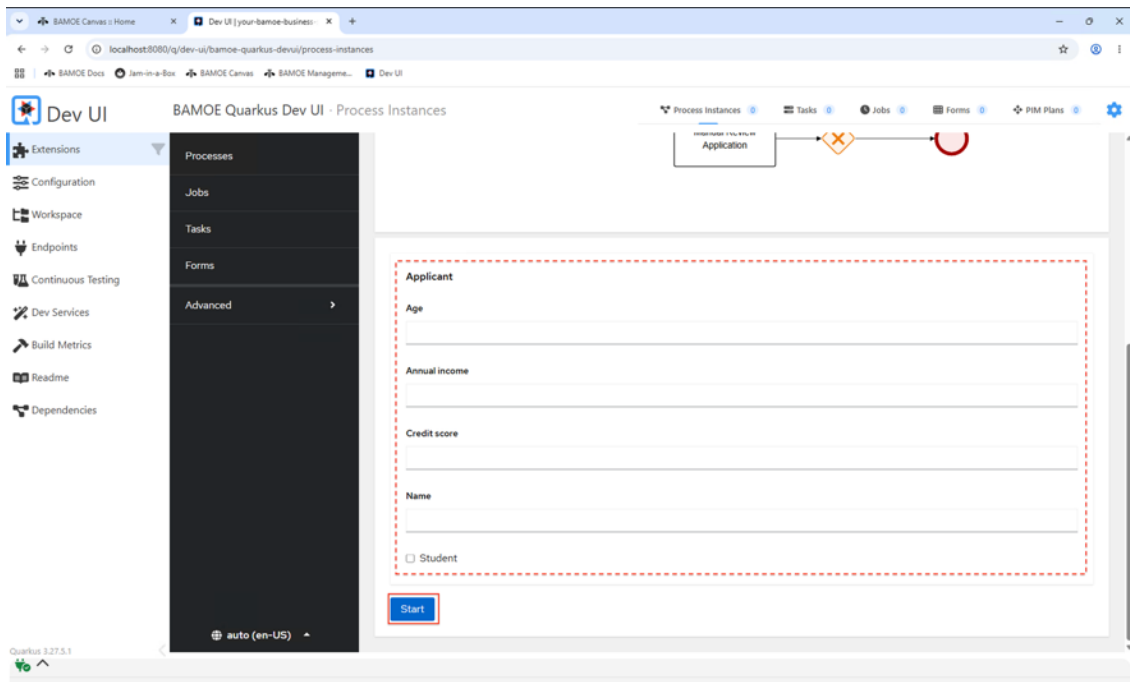


- We will use this development area to start the workflow and monitor the results.
- Navigate now to "Process Definitions" and click the button available in actions.



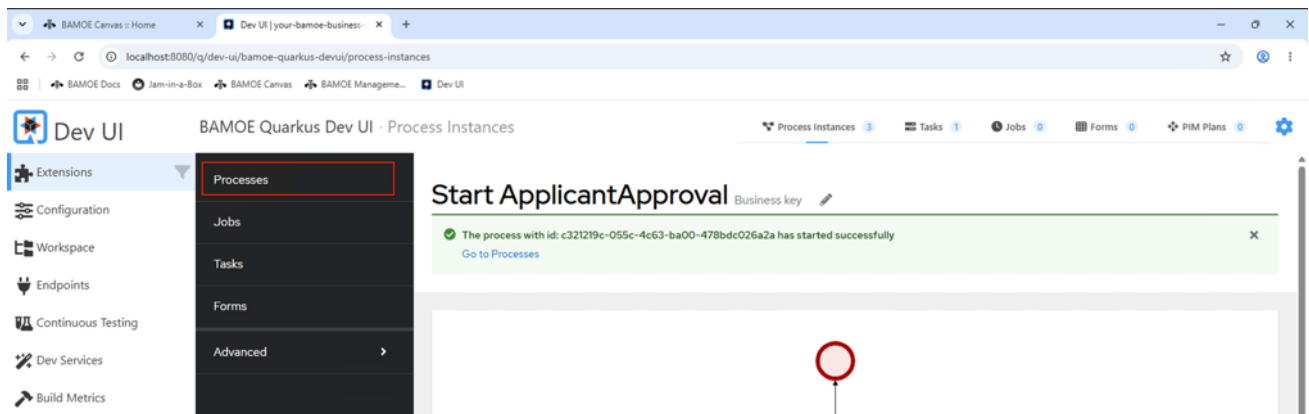
- Navigate to the form and execute the 3 scenarios we covered in the workflow:



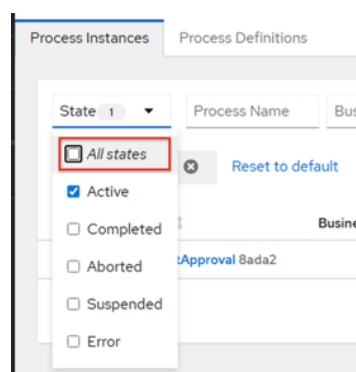


| AGE | ANNUAL INCOME | CREDIT SCORE | IS STUDENT | SCENARIO            |
|-----|---------------|--------------|------------|---------------------|
| 25  | 15000         | 750          | NO         | Automatic approval  |
| 17  | 15000         | 750          | NO         | Automatic rejection |
| 20  | 19000         | 600          | NO         | Manual Review       |

- Return to the top of the page and go to "Processes".



- In "process instances", change the filter to show all states.



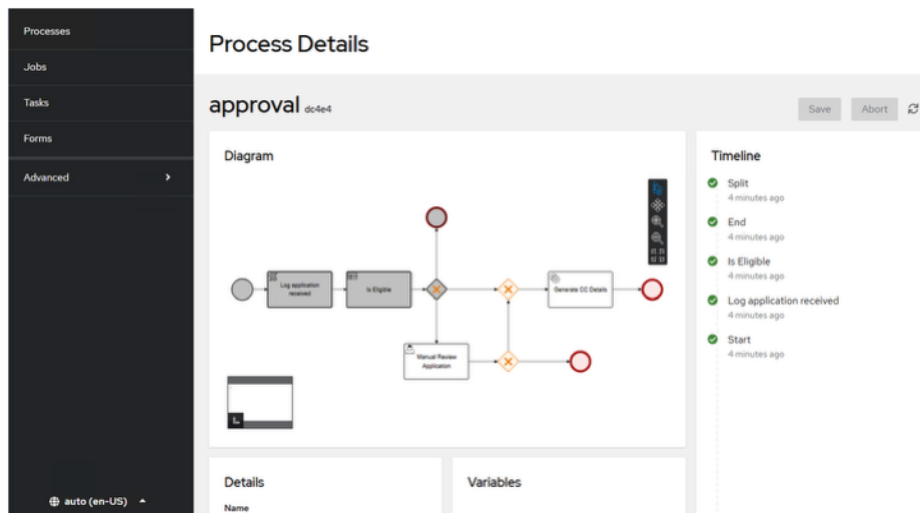
- You should also get this result:
  - 2 processes were "Completed" automatically; these are the automatic Approval and Rejection cases.
  - The other one that is Active is the case for manual approval.

| Process Instances        |                | Process Definitions |              |                  |                        |               |               |     |
|--------------------------|----------------|---------------------|--------------|------------------|------------------------|---------------|---------------|-----|
| State 5                  |                | Process Name        | Business key | Process Insta... |                        |               |               |     |
| State Active X           |                | Completed X         | Aborted X    | 2 more           | Reset to default       |               |               |     |
| <input type="checkbox"/> | Name           | Version             | Busine...    | State            | Endpoint               | Started       | Updated       | SLA |
| <input type="checkbox"/> | approval c3212 | 1.0                 |              | Active           | http://localhost:8080/ | a minute ago  | a minute ago  |     |
| <input type="checkbox"/> | approval dc4e4 | 1.0                 |              | Completed        | http://localhost:8080/ | 2 minutes ago | 2 minutes ago |     |
| <input type="checkbox"/> | approval 46b0a | 1.0                 |              | Completed        | http://localhost:8080/ | 3 minutes ago | 3 minutes ago |     |
| Items per page           |                |                     |              |                  |                        |               |               | 20  |

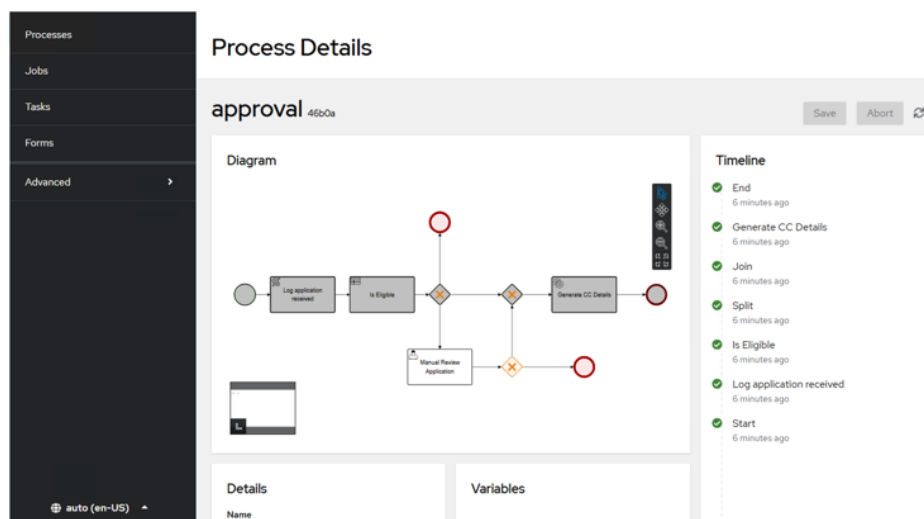
- Click on each one to see the details of each instance.
  - Manual Approval:



- Automatic Rejection:



- Automatic Approval:



- We will simulate the user's action to answer the human task. Click on the Task button.
- Notice that we have a task for the user that we defined. Access it to respond.

Processes

Jobs

**Tasks** 1

Forms

Advanced >

## Tasks jdoe 2

States: 3 Task name Task ID Process Name Process Instance ID

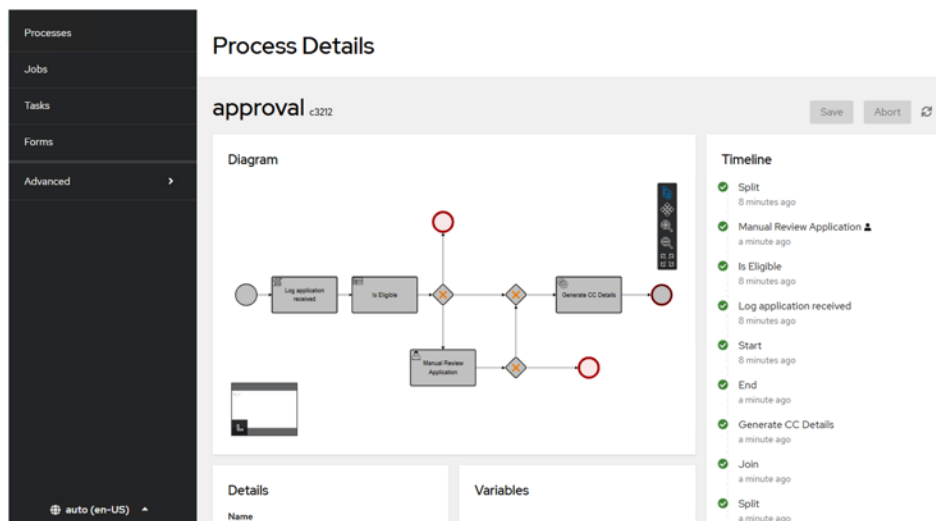
States: Ready X Reserved X In Progress X [Reset to default](#)

| Name                                                                                      | Process Na...     | Process... | Process... | Priority | State      | Started       | Updated         |
|-------------------------------------------------------------------------------------------|-------------------|------------|------------|----------|------------|---------------|-----------------|
| <span style="border: 1px solid red; padding: 2px;">Manual Review Application a3e3e</span> | ApplicantApproval | 1.0        | c3212      |          | ⊙ Reserved | 5 minutes ago | 🕒 5 minutes ago |

Items per page 20

- Remember that in this case we have two possible responses: "approved" or "rejected". You can test the behavior in both cases.

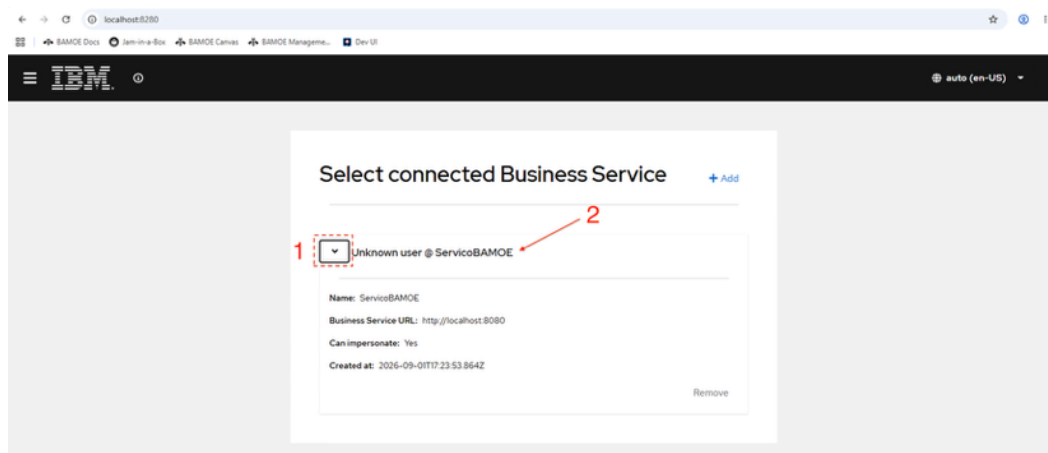
- Go back to the processes, you'll see that the active instance has already been completed.



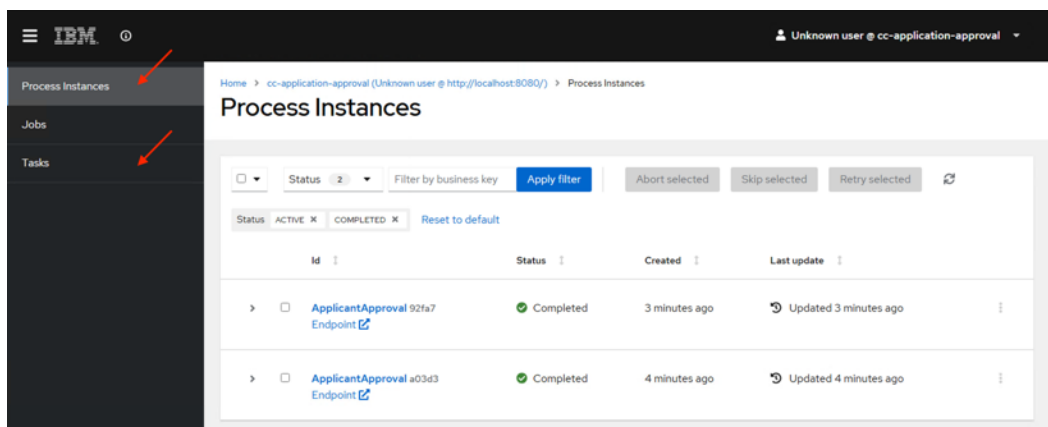
## 7 Exercise 4: Connect your project to the BAMOE Management Console

You can also connect your application to the BAMOE Management Console. Open your Chrome browser. Go to <http://localhost:8280/>

- Right on the first page of the Management Console, you will already have the localhost:8080 port mapped.



- As soon as you access it, you will be taken to the main page, and the side menu will give you access to the Process and Tasks we saw in the previous exercise.



Now you can experiment with how the user can perform their tasks, try creating a new instance through Dev-UI or making a call through Swagger, and see the result in the Management Console. Access the [IBM documentation](#) for more information.

**Congratulations!** You have completed this lab, where we explored a practical use case with Workflow and Decision. I hope that from now on you will be able to create and explore your own cases and other BAMOE features.

## 8 Consult Documentation and Communities

- [IBM BAMOE Official Documentation](#)
- [IBM Business Automation Community: Open Editions](#)